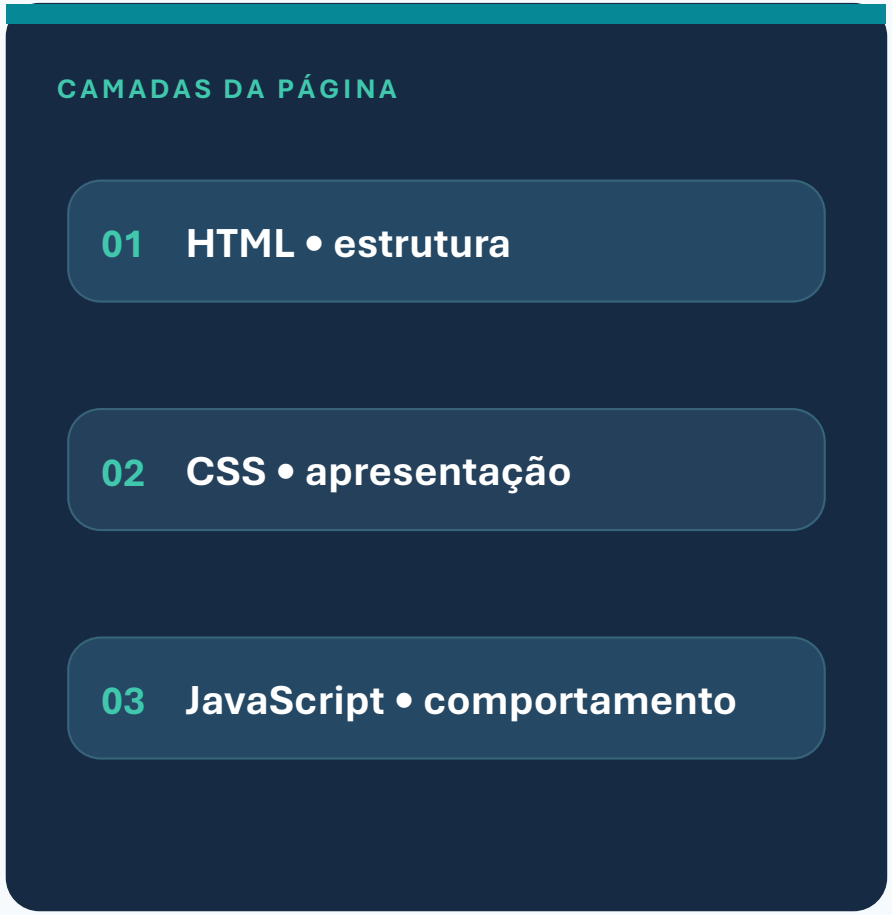
Programação para Internet

Disciplina: Programação para Internet.

Conteúdo

8: fundamentos de programação *front-end*, incluindo variáveis, funções, estruturas de controle e eventos. Conteúdo

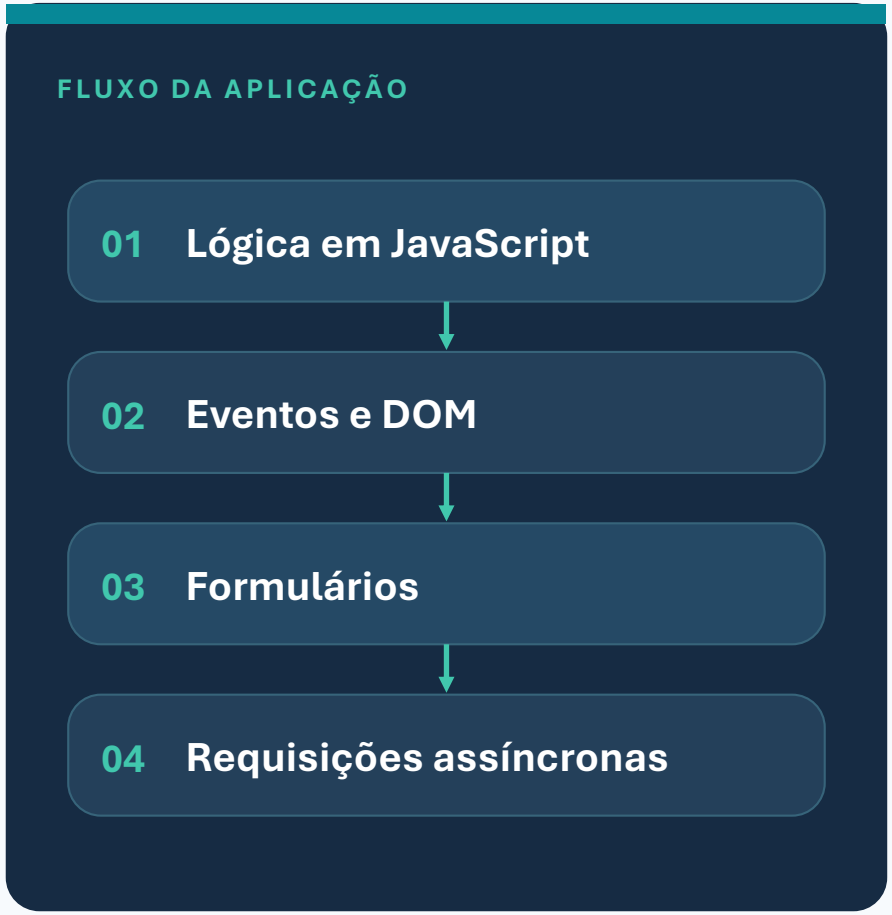
9: manipulação do documento, validação no cliente e comunicação assíncrona básica. A aula acompanha a construção de uma página interativa, relacionando conceitos e resultados observáveis no navegador.



O que vamos aprender?

EM FOCO

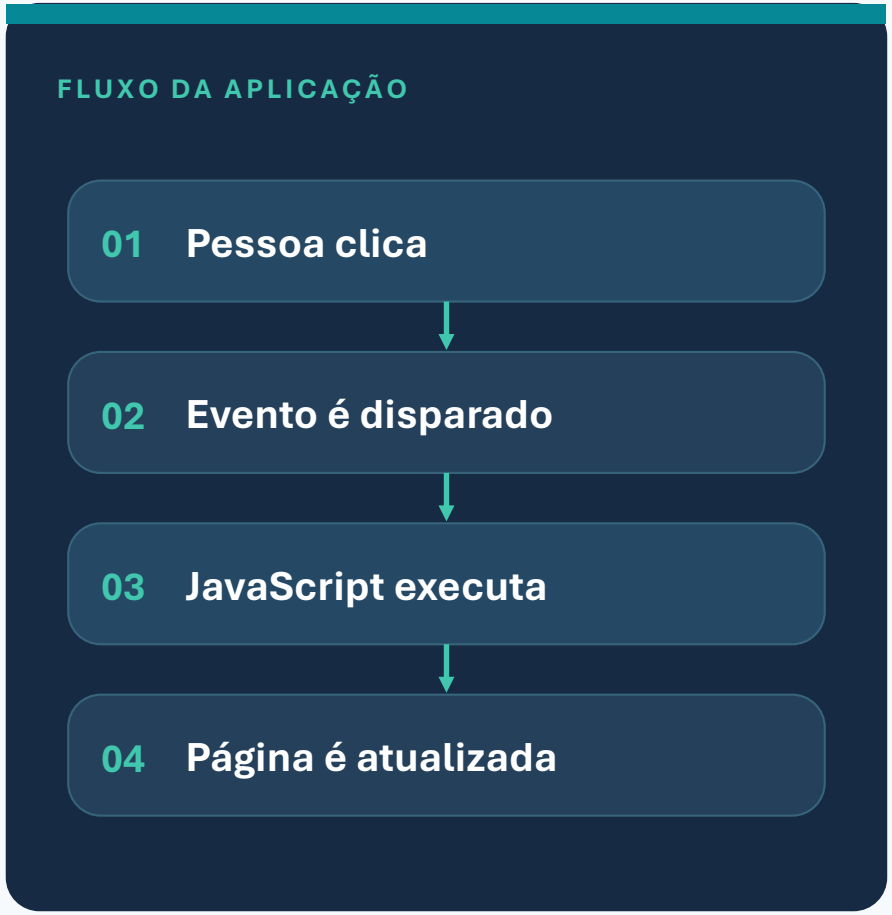
Começaremos com a lógica básica de *JavaScript*, usando valores, decisões, repetições e funções. Depois, estudaremos eventos e modificações de elementos *HTML*. Por fim, construiremos um formulário validado e faremos uma consulta assíncrona. Cada etapa apresenta um pequeno exemplo que será reaproveitado na atividade prática.



O papel do navegador

EM FOCO

O navegador interpreta o *HTML* e o *CSS* e executa *JavaScript* para responder às interações. Um clique pode disparar uma função, que lê informações da página e altera algum elemento. Uma consulta pela rede pode trazer dados novos. Esse ciclo explica por que o navegador é um ambiente de execução e teste.



Ambiente de desenvolvimento

EM FOCO

Para acompanhar a aula, use um editor como *Visual Studio Code* e um navegador moderno. Crie uma pasta com arquivos *index.html*, *styles.css* e *app.js*. Abra as ferramentas de desenvolvedor com *F12* para consultar o *Console*. Um servidor *HTTP* local será usado na etapa de consulta com *fetch*.



Onde entra o JavaScript?

EM FOCO

HTML descreve os elementos de uma página, enquanto *CSS* define sua aparência. *JavaScript* acrescenta decisões e ações: mostrar uma mensagem, validar um formulário ou carregar dados. É recomendável manter o código em um arquivo separado e usar o atributo *defer* para iniciar sua execução quando o documento estiver disponível.

Ligação do arquivo

```
<head>
  <script src="app.js"
          defer></script>
</head>

// Comportamento em app.js
```

Código exemplificativo • elementos editáveis

Variáveis: const e let

EM FOCO

Uma variável associa um nome a um valor que o programa utilizará. Em *JavaScript* moderno, *const* é adequada quando não haverá nova atribuição ao identificador; *let* atende valores que mudam. Use nomes significativos e evite *var* em exemplos novos, pois seu comportamento de escopo é menos intuitivo para iniciantes.

Declarações

```
const curso = "Programação";
let inscritos = 0;

inscritos += 1;
// curso = "Outro";  // erro
```

Código exemplificativo • elementos editáveis

Tipos de dados e conversão

EM FOCO

Valores podem representar texto, números, verdadeiro ou falso, entre outros tipos. Campos de formulários geralmente devolvem texto, mesmo quando parecem números. Antes de calcular, converta conscientemente a entrada e verifique se o resultado é válido. Essa distinção evita resultados inesperados, como concatenar dois números em vez de somá-los.

Texto x número

```
const valor = "10";
const total = Number(valor)
+ 5;

console.log(total); // 15
console.log("10" + 5); //
"105"
```

Código exemplificativo • elementos editáveis

Expressões e operadores

EM FOCO

Operadores ajudam a construir regras de negócio. Podemos somar valores, comparar informações ou combinar condições. Para testar igualdade em *JavaScript*, prefira a comparação estrita, que considera tipo e valor. O operador lógico E exige que as duas condições sejam verdadeiras; o OU aceita pelo menos uma condição verdadeira.

Comparações

```
const idade = 19;
const inscrito = true;

idade >= 18 && inscrito
// Resultado: true

"5" === 5  // false
```

Código exemplificativo • elementos editáveis

Funções organizam tarefas

EM FOCO

Uma função agrupa instruções que executam uma tarefa específica. Ela pode receber parâmetros, processar dados e retornar um resultado. Ao criar funções pequenas, reduzimos repetições e facilitamos testes. No exemplo, a mesma regra de cálculo pode ser chamada várias vezes sem que a fórmula seja escrita novamente.

Função com retorno

```
function calcularTotal(qtd,
preco) {
  return qtd * preco;
}

const total =
calcularTotal(3, 20);
// total vale 60
```

Código exemplificativo • elementos editáveis

Funções de seta e retorno

EM FOCO

Funções também podem ser escritas com a sintaxe de seta. Essa forma é comum em funções curtas e em tratamentos de eventos. Quando o corpo contém uma única expressão, podemos retornar seu valor de maneira implícita. Para alunos iniciantes, vale comparar essa forma com a declaração tradicional antes de adotá-la.

Duas formas equivalentes

```
function dobrar(n) {
  return n * 2;
}

const dobro = n => n * 2;
console.log(dobro(4)); // 8
```

Código exemplificativo • elementos editáveis

Decisões com if e else

EM FOCO

Uma estrutura condicional seleciona caminhos diferentes conforme o resultado de uma expressão lógica. Em formulários, ela permite exibir mensagens adequadas a entradas válidas e inválidas. A decisão é avaliada durante a execução do programa. Assim, a mesma função pode produzir respostas diferentes dependendo dos dados recebidos.

Decisão

```
if (nota >= 7) {
  mensagem = "Aprovado";
} else {
  mensagem = "Revisar";
}
```

Código exemplificativo • elementos editáveis

Repetição com for

EM FOCO

Laços de repetição executam instruções enquanto uma regra de continuidade for atendida. O laço for reúne inicialização, condição e atualização do contador. Ele é útil para percorrer posições de uma lista, produzir várias linhas ou calcular totais. É importante evitar condições que nunca terminam a execução.

Contador

```
for (let i = 1; i <= 3; i++)
{
  console.log(`Item ${i}`);
}

// Item 1
// Item 2
// Item 3
```

Código exemplificativo • elementos editáveis

Listas e objetos

EM FOCO

Um *array* armazena uma sequência ordenada de valores, enquanto um objeto reúne propriedades nomeadas. Esses formatos aparecem em formulários, respostas *JSON* e listas apresentadas na tela. Com *forEach*, podemos executar uma função para cada item. Entender essas estruturas prepara o caminho para receber e exibir dados vindos de uma consulta.

Estruturas de dados

```
const turmas = ["Manhã",
"Noite"];
const curso = { titulo: "JS",
vagas: 25 };

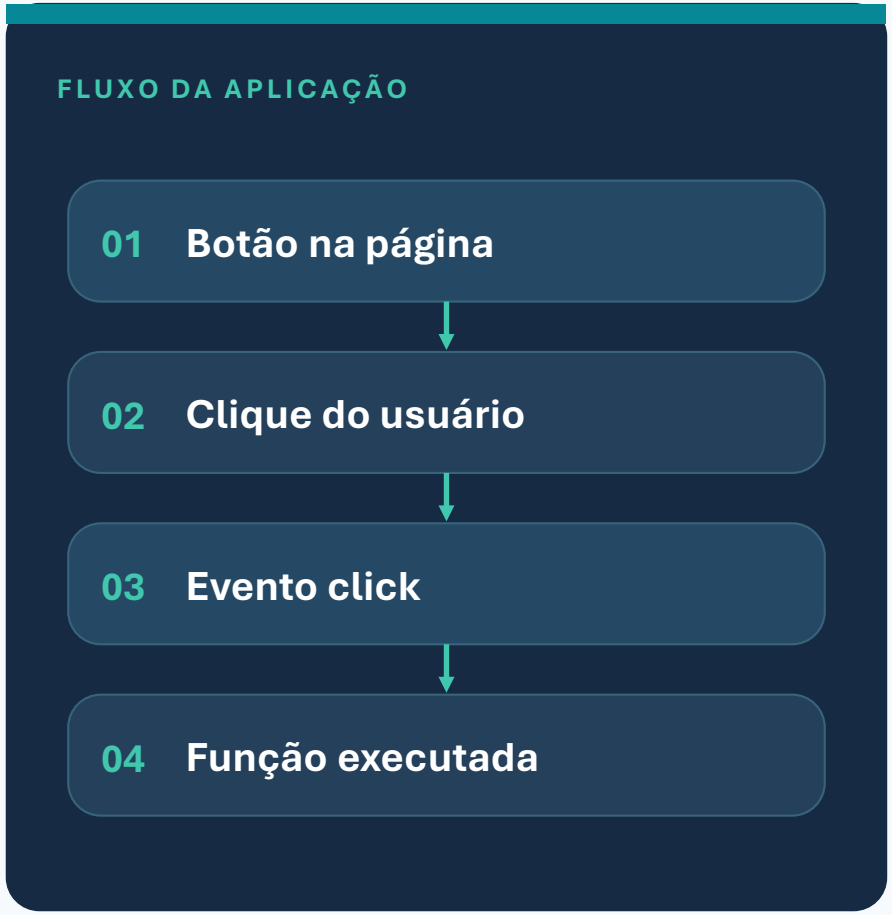
turmas.forEach(t => {
  console.log(t);
});
```

Código exemplificativo • elementos editáveis

O que são eventos?

EM FOCO

Evento é um sinal de que algo ocorreu no navegador, como um clique, uma digitação ou o envio de formulário. O *JavaScript* registra uma função para reagir a esse sinal. Assim, não precisamos verificar continuamente se o usuário clicou: o navegador chama o tratamento quando a interação acontece.



Ouvindo um clique

EM FOCO

O método addEvent*Listener* associa um tipo de evento a uma função. No exemplo, localizamos o botão e registramos uma ação para o clique. Sempre que a pessoa interage, a função executa. Isso mantém o comportamento no arquivo *JavaScript*, em vez de escrever instruções diretamente nos atributos *HTML*.

Ouvinte de evento

```
const botao = document
  .querySelector("#enviar");

botao.addEventListener("click
", () => {
  console.log("Clicou!");
});
```

Código exemplificativo • elementos editáveis

Evento submit e preventDefault

EM FOCO

Um formulário possui comportamento padrão de envio e navegação. Ao registrar um tratamento para *submit*, recebemos um objeto que descreve o evento. Com *preventDefault*, impedimos o comportamento padrão para realizar uma ação didática no próprio navegador. Essa escolha não substitui o envio real quando uma aplicação precisa persistir informações.

Interceptando envio

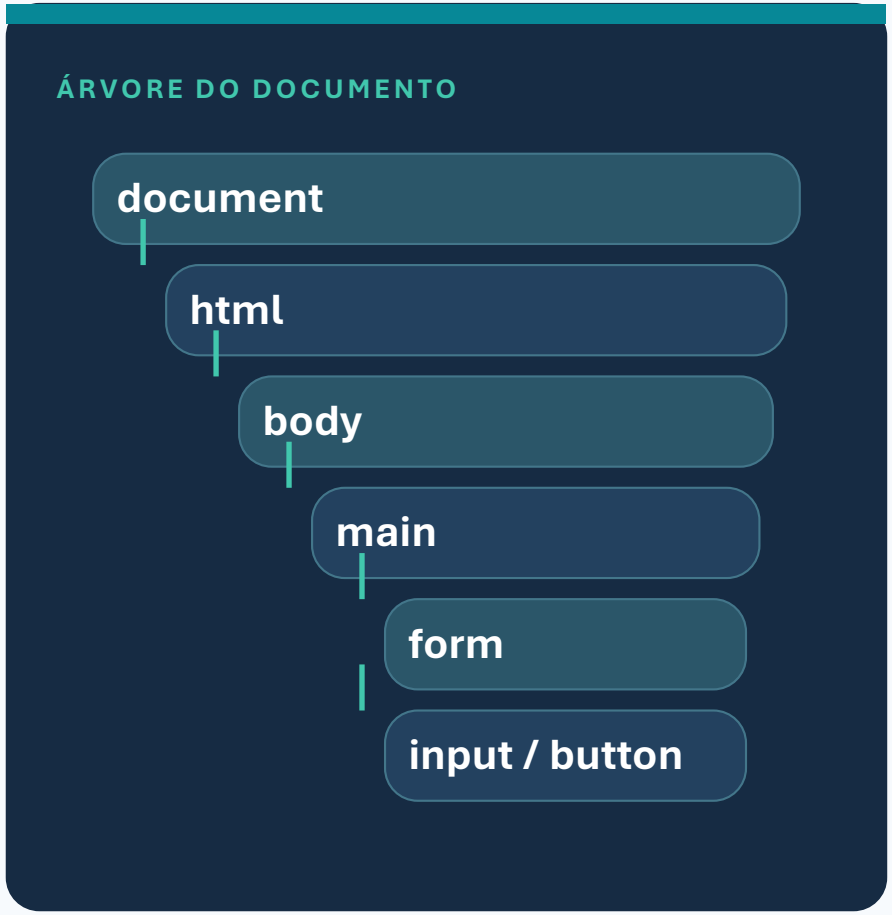
```
form.addEventListener("submit", ev => {
  ev.preventDefault();
  console.log("Processar dados");
});
```

Código exemplificativo • elementos editáveis

DOM: a página como árvore

EM FOCO

O Modelo de Objeto de Documento, ou *DOM*, representa a página em uma estrutura de nós relacionados. Elementos *HTML* ocupam posições nessa árvore. *JavaScript* pode localizar, consultar, criar e alterar nós. Quando modificamos o *DOM*, a mudança normalmente se torna visível sem precisar substituir todo o arquivo *HTML*.



Selecionando elementos

EM FOCO

Para modificar uma página, primeiro precisamos encontrar o elemento desejado. *querySelector* aceita seletores *CSS* e retorna a primeira correspondência. Já *querySelectorAll* devolve uma coleção de elementos encontrados. Em ambos os casos, a busca depende da estrutura *HTML* e da escrita correta do seletor usado.

Seletores

```
const titulo = document
  .querySelector("h1");
const campo = document
  .querySelector("#nome");
const itens = document
  .querySelectorAll(".item");
```

Código exemplificativo • elementos editáveis

Alterando textos com segurança

EM FOCO

Depois de localizar um elemento, podemos alterar seu conteúdo textual. A propriedade *textContent* interpreta o valor como texto, sem processá-lo como marcação *HTML*. Já *innerHTML* interpreta conteúdo como *HTML* e exige muito cuidado com dados não confiáveis. Para mensagens digitadas pelo usuário, preferimos *textContent* por reduzir riscos de injeção.

Atualização segura

```
const mensagem = document

.querySelector("#mensagem");

mensagem.textContent =
  "Inscrição concluída!";
```

Código exemplificativo • elementos editáveis

Alterando classes visuais

EM FOCO

Além do texto, *JavaScript* pode alterar classes *CSS* para informar um estado da interface. Uma classe de sucesso, por exemplo, muda a aparência da mensagem após o envio. A lógica continua no *JavaScript*, enquanto as cores e bordas ficam definidas no *CSS*. Essa separação torna o projeto mais organizado.

Classes CSS

```
mensagem.classList.add("sucesso");
mensagem.classList.remove("erro");

// No CSS:
.sucesso { color: green; }
```

Código exemplificativo • elementos editáveis

Criando novos elementos

EM FOCO

Uma lista pode crescer conforme a pessoa cadastra itens. Em vez de escrever novos elementos manualmente no *HTML*, usamos *createElement* para gerar um nó e *append* para inseri-lo na árvore. Ao preencher *textContent*, tratamos a entrada como texto. Esse padrão aparece em listas, comentários e registros temporários.

Lista dinâmica

```
const item =
document.createElement("li")
;
item.textContent = "Ana •
Noite";

document.querySelector("#lis
ta")
  .append(item);
```

Código exemplificativo • elementos editáveis

Lendo dados de formulários

EM FOCO

Campos de entrada possuem a propriedade value, que permite ler o que foi preenchido. A seleção de uma turma também é obtida por value. Para nomes, *trim* remove espaços em branco nas extremidades. Antes de utilizar os dados, devemos verificar obrigatoriedade, formato e regras específicas da atividade.

Leitura de campos

```
const nome = document

.querySelector("#nome").va
lue.trim();
const email = document

.querySelector("#email").v
alue;
```

Código exemplificativo • elementos editáveis

Validação nativa do HTML

EM FOCO

O *HTML* oferece recursos importantes para validar campos antes do envio. Os atributos *required*, *minlength*, *maxlength* e *type=email* expressam restrições simples. O navegador pode bloquear o envio de um formulário inválido e mostrar mensagens. Ainda assim, é necessário avaliar requisitos específicos que não são atendidos somente com essas opções.

Restrições básicas

```
<input id="nome" required
       minlength="3">

<input id="email"
type="email"
       required>
```

Código exemplificativo • elementos editáveis

Validação personalizada

EM FOCO

Uma regra personalizada pode exigir que o nome tenha pelo menos três caracteres significativos. A função *setCustomValidity* permite definir uma mensagem específica de erro. Quando o problema for corrigido, devemos limpar a mensagem personalizada. O método *reportValidity* mostra o resultado da validação para o usuário durante a interação.

Regra adicional

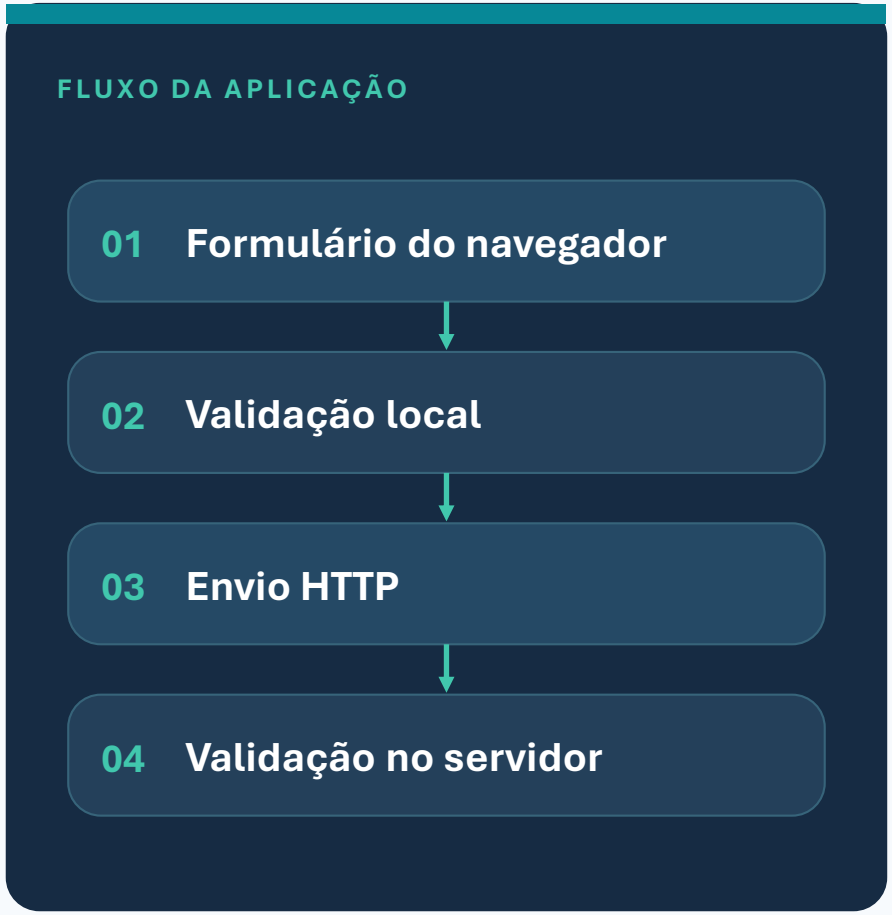
```
const valido =
nome.value.trim().length >=
3;
nome.setCustomValidity(
  valido ? "" : "Nome muito
curto"
);
form.reportValidity();
```

Código exemplificativo • elementos editáveis

Validação no cliente e servidor

EM FOCO

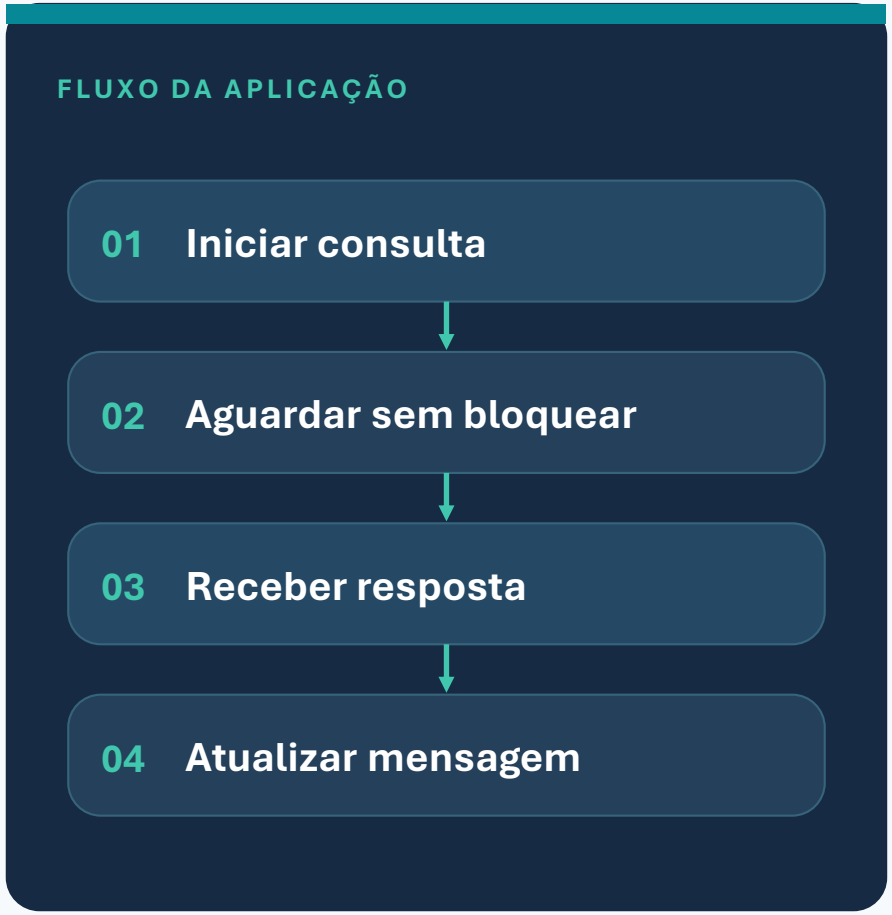
A validação executada no navegador oferece resposta rápida e ajuda a reduzir erros de preenchimento. Porém, uma pessoa pode modificar o *HTML*, desativar scripts ou enviar requisições diretamente. Por essa razão, todo sistema real precisa repetir as validações importantes no servidor, antes de registrar dados ou executar operações.



Por que código assíncrono?

EM FOCO

Uma comunicação com outro computador pode demorar ou falhar. Se o programa precisasse bloquear a interface até receber uma resposta, a experiência seria ruim. A programação assíncrona permite iniciar uma operação e tratar seu resultado quando estiver disponível. Durante esse intervalo, a página pode continuar respondendo a interações permitidas.



Promise representa um resultado futuro

EM FOCO

Uma *Promise* é um objeto que representa o resultado eventual de uma operação assíncrona. Ela começa pendente e depois pode ser concluída com sucesso ou rejeitada por um problema. Esse conceito permite organizar o que fazer após receber dados. O método *fetch* retorna uma *Promise* que será tratada pelo nosso código.

CONCEITOS-CHAVE

1 Pendente

2 Concluída

3 Rejeitada

A sintaxe async e await

EM FOCO

Em *JavaScript*, *async* declara uma função assíncrona; *await* suspende a continuação daquela função até a resolução de uma *Promise*. Essa escrita facilita ler operações em sequência, sem aninhar muitos tratamentos. É recomendável combinar *await* com *try* e *catch* para lidar com falhas que podem ocorrer durante a operação.

Operação assíncrona

```
async function consultar() {
  try {
    const resposta = await
fetch(url);

console.log(resposta.status);
  } catch (erro) {
    console.error(erro);
  }
}
```

Código exemplificativo • elementos editáveis

Consultas HTTP com fetch

EM FOCO

A função *fetch* inicia uma requisição *HTTP* e devolve uma *Promise*. Um pedido *GET* consulta informações sem intenção de alterá-las. Após receber a resposta, podemos solicitar sua interpretação como *JSON*. Esse é um padrão básico para integrar uma página a uma *API* ou a um arquivo de dados servido por *HTTP*.

GET e JSON

```
const resposta = await
fetch(
  "./dados/curso.json"
);
const curso = await
resposta.json();
console.log(curso.titulo);
```

Código exemplificativo • elementos editáveis

Verificando o status HTTP

EM FOCO

Uma requisição pode terminar com uma resposta de erro, como 404 para recurso não encontrado ou 500 para falha interna do servidor. *Fetch* não rejeita automaticamente uma *Promise* apenas porque recebeu um desses códigos. Precisamos consultar *response.ok* e produzir um erro controlado, evitando tratar uma resposta malsucedida como sucesso.

Verificação de status

```
const resposta = await
fetch(url);

if (!resposta.ok) {
  throw new Error(`HTTP
${resposta.status}`);
}

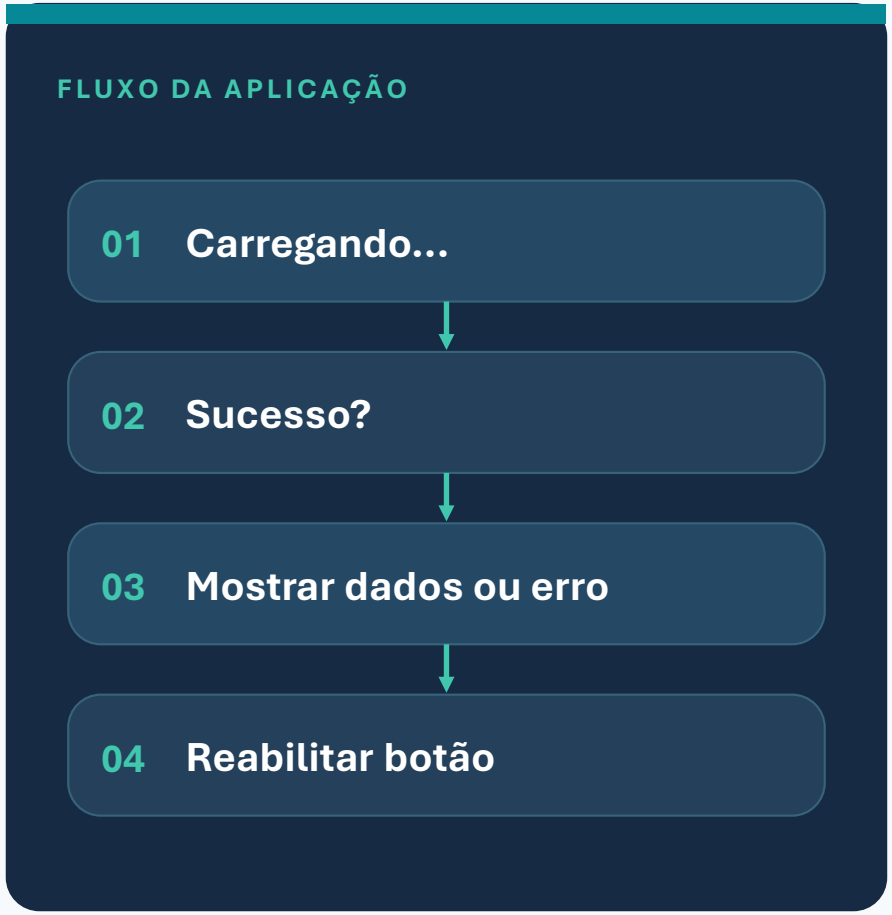
const dados = await
resposta.json();
```

Código exemplificativo • elementos editáveis

Estados de carregamento e falha

EM FOCO

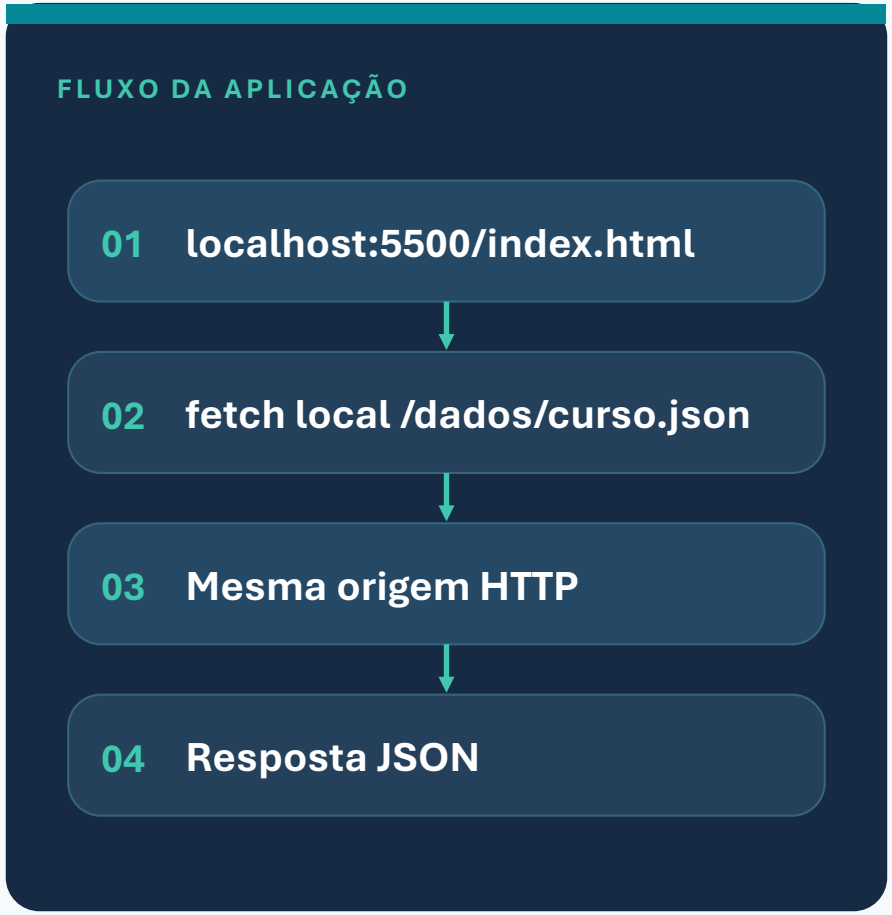
Uma interface deve informar quando a consulta está em andamento, quando foi bem-sucedida e quando falhou. Podemos desabilitar temporariamente o botão, escrever uma mensagem e reabilitá-lo ao final. O bloco *finally* é útil porque executa tanto após sucesso quanto após erro, evitando que a tela fique bloqueada indevidamente.



Origem, servidor local e CORS

EM FOCO

Requisições entre origens diferentes podem ser limitadas pelo navegador por motivos de segurança. O mecanismo *CORS* permite que servidores autorizem determinados acessos. Para o laboratório, utilizaremos um arquivo *JSON* na mesma origem da página, fornecido por um servidor *HTTP* local. Assim, evitamos dependência de serviços externos e problemas desnecessários de permissão.



Testes com ferramentas do navegador

EM FOCO

O *Console* mostra mensagens e erros de *JavaScript*, enquanto a aba Rede apresenta solicitações *HTTP*, códigos de resposta e arquivos transferidos. Ao investigar um defeito, primeiro reproduza o comportamento e observe o que aconteceu. Usar essas ferramentas transforma suposições em evidências e ajuda a identificar problemas com rapidez.



Laboratório: arquivos e objetivo

EM FOCO

Construiremos uma página fictícia para inscrições em oficina, com campos de nome, e-mail e turma. Ao enviar, o sistema verificará os dados e criará um item na lista. Outro botão consultará informações de uma oficina em um arquivo *JSON*. O projeto usa apenas *HTML*, *CSS*, *JavaScript* e um servidor local.



Prática 1: iniciar o servidor

EM FOCO

Extraia o laboratório e abra um terminal na pasta onde está o arquivo *index.html*. Execute *python* -m *http*.server 5500, ou py -m *http*.server 5500 no Windows quando necessário. Em seguida, abra *http://localhost*:5500 no navegador. Confirme que a página aparece e que nenhum erro é mostrado no *Console*.

Passos no terminal

```
# Terminal na pasta do
projeto
python -m http.server 5500

# No navegador
http://localhost:5500

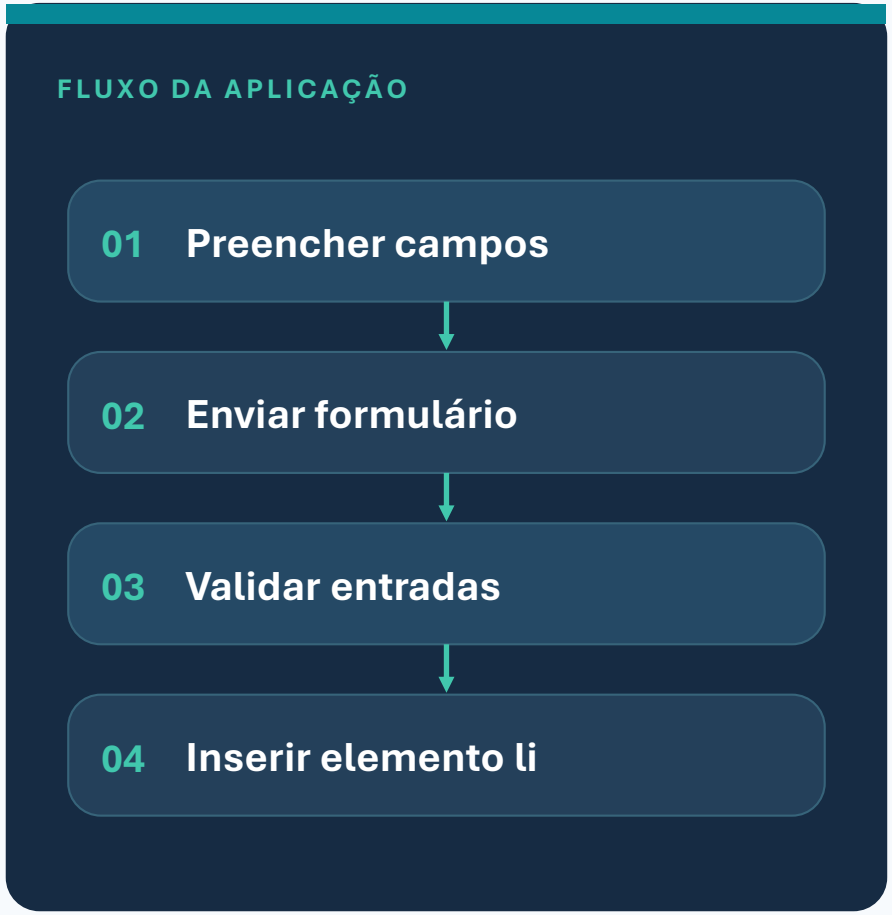
# Abra o console com F12
```

Código exemplificativo • elementos editáveis

Prática 2: observar eventos

EM FOCO

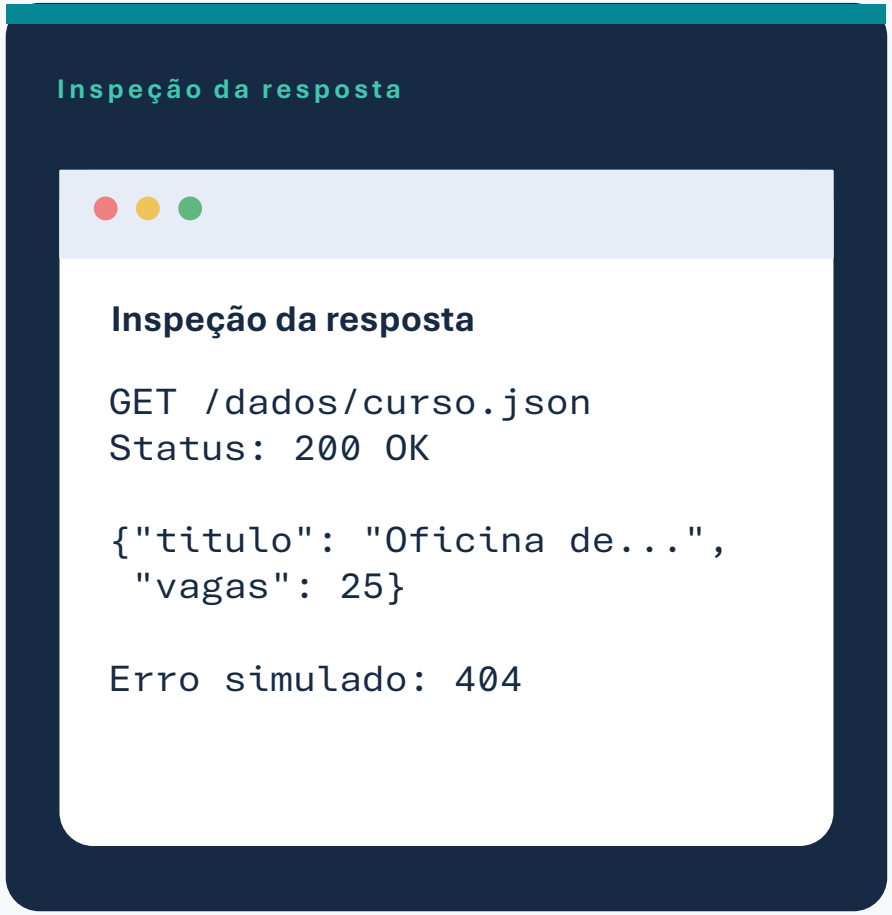
Abra *app.js* e localize o tratamento de *submit*. Preencha os campos com dados fictícios e envie uma inscrição válida. Observe como *preventDefault* evita a navegação e como a função cria um elemento de lista. Repita com entradas inválidas para comparar o comportamento e identificar as condições de aceitação.



Prática 3: acompanhar o fetch

EM FOCO

Clique em Consultar oficina e acompanhe a mensagem de carregamento. Abra a aba Rede do navegador e procure a solicitação *curso.json*. Observe o *status* 200 e o conteúdo retornado. Depois, simule uma falha alterando temporariamente o endereço do arquivo e verifique a mensagem de erro antes de restaurar o caminho correto.



Exercício resolvido: situação

EM FOCO

Uma página de cadastro deve receber nome e turno, impedir nomes com menos de três caracteres significativos e mostrar uma mensagem de confirmação sem recarregar a tela. Para resolver, precisamos selecionar os campos, interceptar o envio, testar a regra e atualizar o conteúdo de um elemento. O próximo slide apresenta a lógica.



Exercício resolvido: código

EM FOCO

A solução combina conhecimentos de diferentes partes da aula. O evento *submit* inicia a operação, *preventDefault* evita o envio automático, *trim* elimina espaços externos e a condição decide a mensagem. O uso de *textContent* insere um texto literal na página. É uma solução didática local, sem gravação em servidor.

Solução essencial

```
form.addEventListener("submit
", ev => {
  ev.preventDefault();
  const nome =
campo.value.trim();
  saida.textContent =
nome.length < 3
    ? "Nome inválido"
    : `Confirmado: ${nome}`;
});
```

Código exemplificativo • elementos editáveis

Exercício avaliativo

EM FOCO

A partir do laboratório, acrescente o campo idade e impeça inscrições quando a idade informada for menor que dezesseis anos. Atualize a mensagem para incluir a turma selecionada. Na consulta *JSON*, mostre também a duração do curso. Entregue os arquivos alterados e explique, em poucas linhas, quais funções e validações foram utilizadas.

CONCEITOS-CHAVE

1 HTML: novo campo

2 JS: validar idade

3 DOM: mensagem

4 JSON: exibir duração

Checklist de aprendizagem

EM FOCO

Ao finalizar, verifique se consegue distinguir *const* de *let*, criar funções, escrever condições, percorrer listas e registrar eventos. Confirme também que sabe selecionar elementos *HTML*, validar entradas e atualizar mensagens. Por último, explique como *fetch*, *await*, *response.ok*, *catch* e *finally* colaboram para consultas assíncronas confiáveis e compreensíveis.

AUTOAVALIAÇÃO

- ✓ Lógica e funções
- ✓ Eventos e DOM
- ✓ Validação
- ✓ HTTP assíncrono

Referências e leituras

EM FOCO

Os conceitos e exemplos foram organizados com apoio da documentação MDN sobre *JavaScript*, Modelo de Objeto de Documento, eventos, validação de formulários e *API Fetch*. Para aprofundar, consulte também a documentação das ferramentas de desenvolvedor do Chrome. Os endereços completos estão disponíveis nas anotações deste slide e no roteiro do laboratório.

FONTES PARA CONSULTA

- MDN • Guia JavaScript
- MDN • DOM e eventos
- MDN • Validação de formulários
- MDN • API Fetch e CORS
- Chrome • DevTools